Ben Gurion University of the Negev
Din Yair Sadot
302211917

Deep Learning from Scratch – Mini Project in Deep Learning

Part I – Classifier and Optimizer

1. Soft-max regression

In this part I have implemented the multinomial logistic (soft-max) regression classifier.

- First, I have implemented the soft-max function itself, which transforms a vector of real-valued numbers into a probability distribution. This process enables classification using the converted raw outputs of the last layer (logits). The soft-max function is implemented according to this formula:

$$\frac{\exp\,(z_i - \eta)}{\sum_{j=1}^{n} \exp\,(z_j - \eta)} \quad , \qquad \eta = \max\,(z)$$

  This formula concerns an input vector z with n features. Subtracting the parameter η is done to enhance numerical stability. It prevents numerical overflow or underflow in the exponential computation and does not affect the ratio.

  Note: the function soft-max which I implemented can be applied to a matrix instead of a vector. When the input is a matrix, Python is applying the function to it column-wise, which is suitable for our case since each column of the input matrix represents data of a single sample.

- Next, I have implemented the cross-entropy loss function according to the next formula:

$$\mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{n} Y_{i,j} * \log\,(\hat{Y}_{i,j})$$

  The cross-entropy loss, and any loss function in general, indicate how well the model's predictions match the true target values. It provides a measure of error that guides the optimization process that we're going to carry out. The equation is computed by multiplying two matrices element-wise, and then summing all the elements of the result.

  Note: the loss computation aggregates the loss calculation of a few samples together, then averages the result by the number of samples used.
  Note: Before the $\log\,(\hat{Y})$ computations I have clipped the values to a small epsilon value to prevent numerical issues concerning log of a very small number, which can be interpreted as log of zero (which outputs $-inf$ and causes unwanted program behavior).

Ben Gurion University of the Negev
Din Yair Sadot
302211917

- Next, I have implemented a function <u>compute-gradients</u> which calculates the derivative of the loss with respect to the weights matrix, and with respect to the bias vector. The computations were calculated according to the following formulas:

(1) $\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{Y}_i}\frac{\partial \hat{Y}_i}{\partial z_i}\frac{\partial z_i}{\partial W}$, $\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial \hat{Y}_i}\frac{\partial \hat{Y}_i}{\partial z_i}\frac{\partial z_i}{\partial b}$

(2) $$\frac{\partial \mathcal{L}}{\partial \hat{Y}_i} = \begin{pmatrix} \frac{\partial(-\sum_{j=1}^{k} y_j * \log(\widehat{y_j}))}{\partial \hat{Y}_{i,1}} \\ \cdots \\ \frac{\partial(-\sum_{j=1}^{k} y_j * \log(\widehat{y_j}))}{\partial \hat{Y}_{i,k}} \end{pmatrix} = -\begin{pmatrix} \frac{y_1}{\hat{Y}_{i,1}} \\ \cdots \\ \frac{y_k}{\hat{Y}_{1,k}} \end{pmatrix} = -Y_i \oslash \hat{Y}_i$$

(3) $$\frac{\partial \hat{Y}_i}{\partial z_i} = \begin{pmatrix} \hat{Y}_{i,1}(1-\hat{Y}_{i,1}) & -\hat{Y}_{i,1}\hat{Y}_{i,2} & \cdots \\ -\hat{Y}_{i,1}\hat{Y}_{i,2} & \cdots & -\hat{Y}_{i,k-1}\hat{Y}_{i,k} \\ \cdots & -\hat{Y}_{i,k-1}\hat{Y}_{i,k} & \hat{Y}_{i,k}(1-\hat{Y}_{i,k}) \end{pmatrix}$$

(4) $\frac{\partial \mathcal{L}}{\partial z_i} = \frac{\partial \mathcal{L}}{\partial \hat{Y}_i}\frac{\partial \hat{Y}_i}{\partial z_i} = \hat{Y}_i - Y_i$ (grad_z in the program)

(5) $\frac{\partial z_i}{\partial W} = \frac{\partial(WX+b)}{\partial W} = X_i$, $\frac{\partial z_i}{\partial b} = \frac{\partial(WX+b)}{\partial b} = (1, \dots, 1)$

(6) $\frac{\partial \mathcal{L}}{\partial W} = \frac{1}{m}(\hat{Y} - Y)X^t$, $\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m}\frac{\partial \mathcal{L}}{\partial z_i} = \frac{1}{m}\sum_{i=1}^{m}(\hat{Y}_i - Y_i)$

<u>Note</u>: the derivatives calculated in (1) – (5) are concerning a single sample. In (6) you can see the averaging of the results of these calculations among the number of samples evaluated. This model would fit our calculations later in SGD.

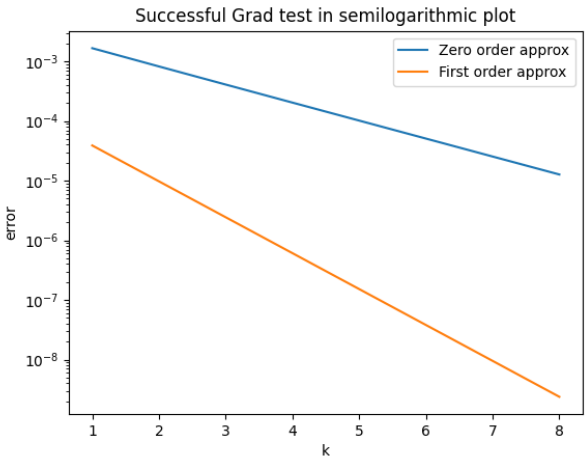
Ben Gurion University of the Negev
Din Yair Sadot
302211917

Gradient verification test

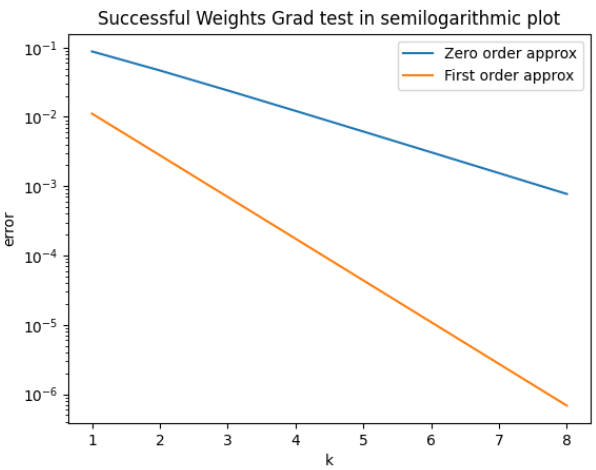
I have tested my derivatives and gradients with the gradient verification test as demonstrated in the notes. I have compared the quantity of two formulas:

- Error order 0: $|f(x+\varepsilon d) - f(x)| = O(\varepsilon)$
- Error order 1: $|f(x+\varepsilon d) - f(x) - \varepsilon d * grad(x)| = O(\varepsilon^2)$

Where d is a vector which is derived from a standard normal distribution, and epsilon is a small number. I have got the following results:



Bias gradient test		
K	Error order 1	Error order 2
1	1.664258e-03	3.893683e-05
2	8.224060e-04	9.745517e-06
3	4.087680e-04	2.437792e-06
4	2.037748e-04	6.096244e-07
5	1.017350e-04	1.524282e-07
6	5.082939e-05	3.810980e-08
7	2.540517e-05	9.527794e-09
8	1.270020e-05	2.381992e-09



Weights gradient test		
K	Error order 1	Error order 2
1	8.791148e-02	1.114975e-02
2	4.673056e-02	2.800050e-03
3	2.406438e-02	7.009250e-04
4	1.220736e-02	1.752969e-04
5	6.147498e-03	4.382922e-05
6	3.084706e-03	1.095772e-05
7	1.545092e-03	2.739469e-06
8	7.732310e-04	6.848714e-07

In both tests the zero order approximation decreases linearly, and the first order approximation decreases quadratically, which means the gradients are indeed correct.

Ben Gurion University of the Negev
Din Yair Sadot
302211917

2. Stochastic Gradient Descent

I have implemented the mini-batch variant of SGD. In this variant in each iteration (or epoch) over the data set the program shuffles the indexes of the samples and picks a random subset of the data set which is called 'batch'. The batch size and the number of iterations are given as parameters to the optimizer.

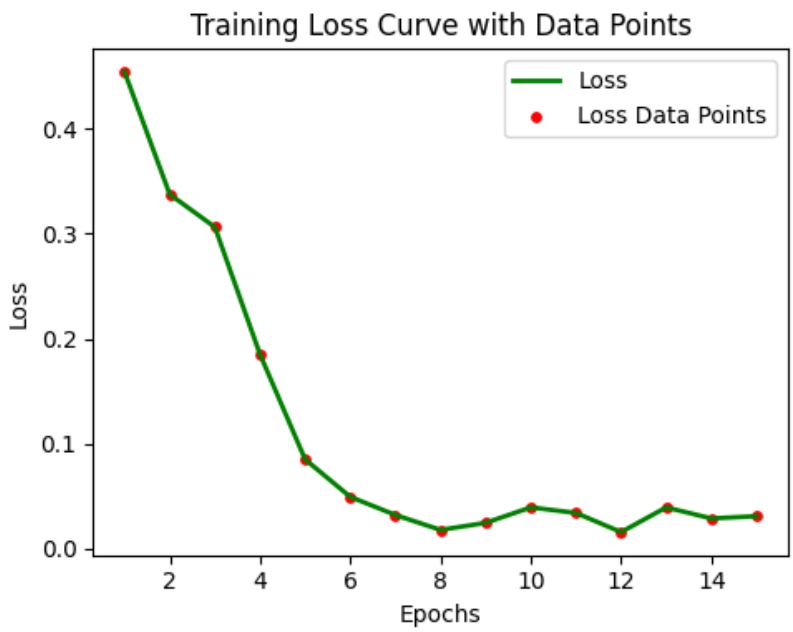
In each epoch this optimizer goes through the whole data set, but instead of computing the gradient of the whole data set at once, the gradient is computed w.r.t the mini batches in a few cycles. The weights and bias are updated upon the process of each batch, and the cross-entropy loss is calculated upon the end of each epoch.

I have tested the SGD variant on a synthetic least squares problem of 15 samples and 4 features (or predictors) for each sample.

The loss function is calculated as: $\mathcal{L}(x) = \frac{1}{2}||Ax - b||_2^2 = \frac{1}{2}(Ax - b)^t(Ax - b)$

The gradient of the loss w.r.t the parameters vector x is: $\nabla_x \mathcal{L}(x) = A^t(Ax - b)$

I initialized the optimizer with randomized values for A and created a synthetic noisy b, and ran the algorithm with a learning rate of 0.01, for 15 epochs, with a batch size of 5, and got the following results:



Epoch	Loss
1	0.4542
2	0.3376
3	0.3063
4	0.1851
5	0.0849
6	0.0492
7	0.0319
8	0.0177
9	0.0244
10	0.0392
11	0.0339
12	0.0156
13	0.0391
14	0.0286
15	0.0308

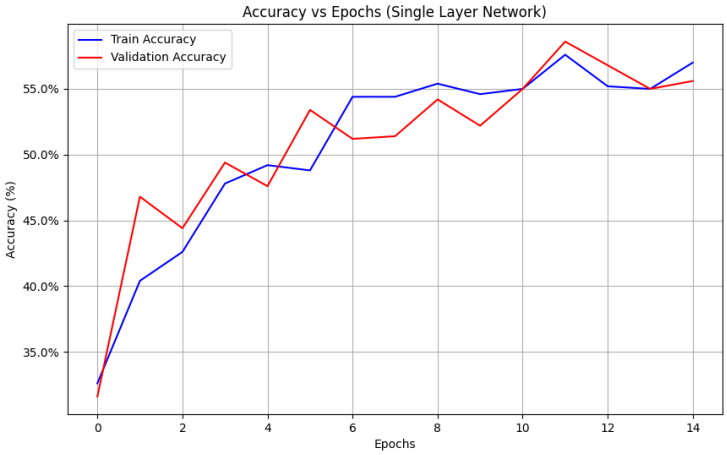
As can be seen from the results, the optimizer has managed to minimize the loss function.

Ben Gurion University of the Negev
Din Yair Sadot
302211917

3. In the last section of this part, I have tested my optimizer on the soft-max classifier. I've used the "Peaks" data set to demonstrate the minimization of the soft-max. The "Peaks" data set has 2-dimensional data points of 5 different classes.

   I have tried many different values for the hyper-parameters to accomplish this task.
   In some of the tries I've done the accuracy has not improved a bit, and in some of them the accuracy kept dropping and rising dramatically.

   Finally, I found that best results we're accomplished when using a batch size of 256 samples, for 15 epochs, using a fixed learning rate of 0.01. I have plotted a graph of the classification success rate in terms of epochs, using a subset of 500 samples per epoch to compute the success percentage, and got the following results:



   As can be seen from the graph, the optimizer has managed to minimize the loss upon the training, but the accuracy is not so good (only about 55% success rate). I assume that one of the causes of the poor success rates is that the network has only one layer for now.

   In the next part of the project, I would implement a deep neural network and classify this data set again, and I hope that the success percentages will rise much higher.

Ben Gurion University of the Negev
Din Yair Sadot
302211917

Part II – The Neural Network

1. In this section I have implemented a standard fully-connected neural network of 5 layers (3 hidden layers), using the tanh activation function. We'll test the derivatives of each layer by showing results of Jacobian tests for each layer.

   Note: the standard approach is that in the backpropagation the derivatives are calculated as the Jacobian transposed times a vector of activations. In this network the derivatives are calculated as vector times Jacobian, since this aligns with the model's parameters' dimensions. It aligns with the fact that the dataset's samples are arranged in columns.

   Note: In this section (Part 2.1), I have computed the derivatives in the backpropagation directly, since it is clearer and easier to understand, and since I can pick the length of the network however I want. In the next sections (2.3 for example), the network is supposed to work with L layers, where L is a parameter, and so the code of the backpropagation would be switched to a loop, instead of manual calculations.

   In order to check each layer's derivatives separately, I have implemented an auxiliary function "jacobian_test_layer_i". This function initializes all the layers of a network but performs a forward pass only from a layer i (i is given as parameter). The function sets the weights matrix and bias vector of a certain layer i. This way we can check the influence of a change in the loss w.r.t a small change in the inputs, while checking that each layer has a proper derivative, sequentially (beginning from the output layer and until the first layer).

   I have verified that the Jacobians and derivatives of all layers are correct by performing the Jacobian test and gradient test for each layer. The process is pretty similar to the gradient verification test I have mentioned in part I, comparing two Taylor's expansions of the loss function with a small quantity of change, and showing a difference in order of descent:

$$\text{I.} \quad \|f(x+\varepsilon d) - f(x)\|_2 = O(\varepsilon)$$
$$\text{II.} \quad \|f(x+\varepsilon d) - f(x) - JacMV(x, \varepsilon d)\|_2 = O(\varepsilon^2)$$

   I'll denote the following elements in the network as follows:

$Z_1 = W_1 * X_{input} + b_1, \qquad A_1 = \tanh(Z_1)$

$Z_2 = W_2 * A_1 + b_2, \qquad A_2 = \tanh(Z_2)$

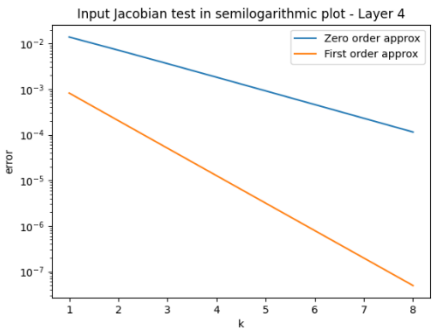
$Z_3 = W_3 * A_2 + b_3, \qquad A_3 = \tanh(Z_z)$

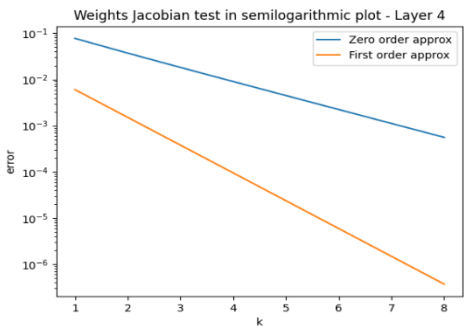
$Z_4 = W_4 * A_3 + b_4, \qquad A_4 = \hat{Y} = \tanh(Z_4)$

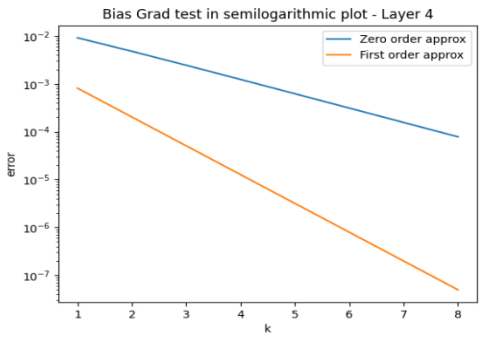
Ben Gurion University of the Negev
Din Yair Sadot
302211917

The derivatives of this network are as follows (verified by plots of Jacobian/Gradient tests):

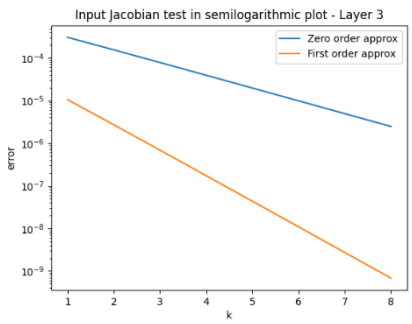
1. $\frac{\partial \mathcal{L}}{\partial Z_4} = \frac{1}{m}\left(\hat{Y} - Y\right)$ (cross-entropy loss calculation averages over m samples)
2. $\frac{\partial \mathcal{L}}{\partial W_4} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial W_4} = \frac{\partial \mathcal{L}}{\partial Z_4} A_3$
3. $\frac{\partial \mathcal{L}}{\partial b_4} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial b_4} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_4}$ (bias vector is averaged over samples)

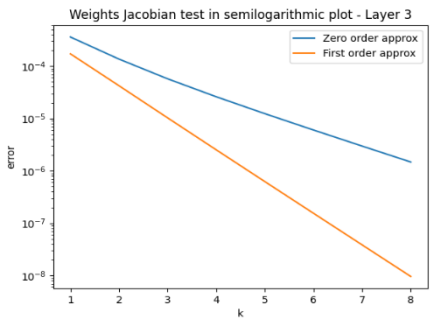


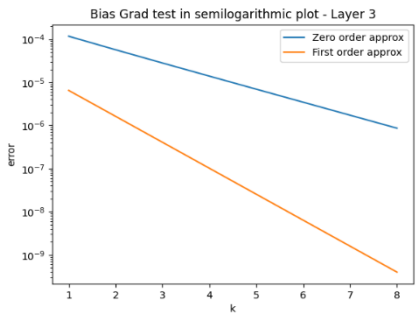




4. $\frac{\partial \mathcal{L}}{\partial A_3} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3} = \frac{\partial \mathcal{L}}{\partial Z_4} W_4 = W_4^t * \frac{\partial \mathcal{L}}{\partial Z_4}$ (aligns with model's matrices shape)
5. $\frac{\partial \mathcal{L}}{\partial Z_3} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3} = \frac{\partial \mathcal{L}}{\partial A_3}(1 - \tanh^2(Z_3))$
6. $\frac{\partial \mathcal{L}}{\partial W_3} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial W_3} = \frac{\partial \mathcal{L}}{\partial Z_3} * A_3$
7. $\frac{\partial \mathcal{L}}{\partial b_3} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial b_3} = \frac{1}{m}\frac{\partial \mathcal{L}}{\partial Z_3}$

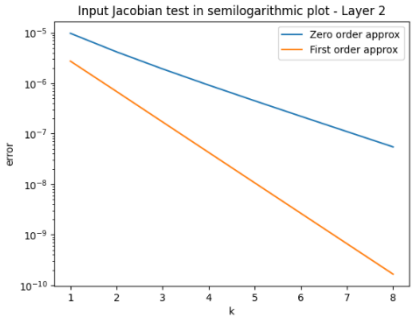


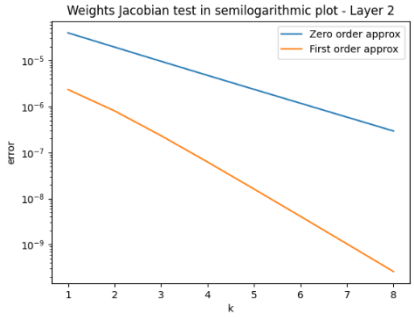


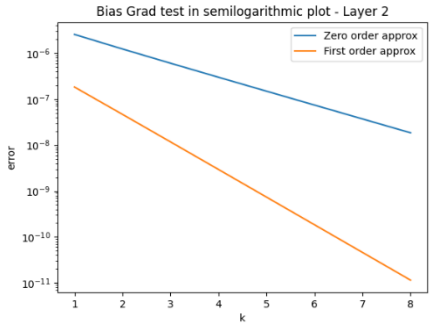


8. $\frac{\partial \mathcal{L}}{\partial A_2} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial A_2} = \frac{\partial \mathcal{L}}{\partial Z_3} W_3 = W_3^t * \frac{\partial \mathcal{L}}{\partial Z_3}$
9. $\frac{\partial \mathcal{L}}{\partial Z_2} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial A_2}\frac{\partial A_3}{\partial Z_2} = \frac{\partial \mathcal{L}}{\partial A_2}(1 - \tanh^2(Z_2))$
10. $\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial A_2}\frac{\partial A_3}{\partial Z_2}\frac{\partial Z_2}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial Z_2} * A_2$
11. $\frac{\partial \mathcal{L}}{\partial b_2} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial A_2}\frac{\partial A_3}{\partial Z_2}\frac{\partial Z_2}{\partial b_2} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_2}$

Ben Gurion University of the Negev
Din Yair Sadot
302211917





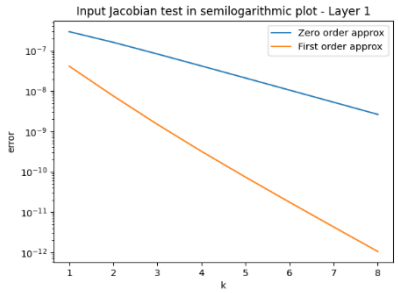


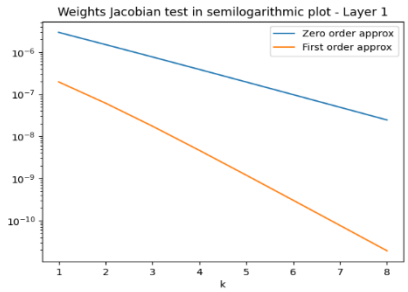
12. $\frac{\partial \mathcal{L}}{\partial A_1} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial A_2}\frac{\partial A_3}{\partial Z_2}\frac{\partial Z_2}{\partial A_1} = \frac{\partial \mathcal{L}}{\partial Z_2} W_2 = W_2^t * \frac{\partial \mathcal{L}}{\partial Z_2}$

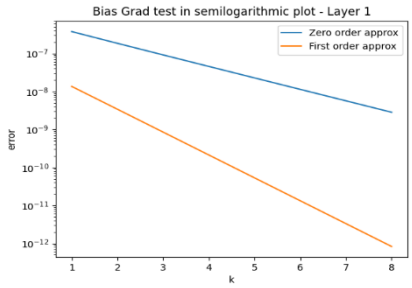
13. $\frac{\partial \mathcal{L}}{\partial Z_1} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial A_2}\frac{\partial A_3}{\partial Z_2}\frac{\partial Z_2}{\partial A_1}\frac{\partial A_1}{\partial Z_1} = \frac{\partial \mathcal{L}}{\partial A_1}(1 - \tanh^2(Z_1))$

14. $\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial A_2}\frac{\partial A_3}{\partial Z_2}\frac{\partial Z_2}{\partial A_1}\frac{\partial A_1}{\partial Z_1}\frac{\partial Z_1}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial Z_1} * A_1$

15. $\frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial Z_4}\frac{\partial Z_4}{\partial A_3}\frac{\partial A_3}{\partial Z_3}\frac{\partial Z_3}{\partial A_2}\frac{\partial A_3}{\partial Z_2}\frac{\partial Z_2}{\partial A_1}\frac{\partial A_1}{\partial Z_1}\frac{\partial Z_1}{\partial b_1} = \frac{1}{m}\frac{\partial \mathcal{L}}{\partial Z_1}$







Ben Gurion University of the Negev
Din Yair Sadot
302211917

ResNet

I'll now repeat the previous section for the residual neural network.

In this section, I have calculated the derivatives using a for loop and not explicitly.

Note: In this section I have tested that each block's derivatives are correct. To prove that the inner layers' derivatives are correct as well I have added two additional tests at the end of this section. Anyhow, the fact that the blocks' derivatives are correct also proves that the inner layers' derivatives are correct.

I have implemented 2 auxiliary functions to perform this test:

- forward_pass_resnet_i – which performs a forward pass from the i'th block of the network. The function also fills the previous activations (before the i'th activation) with random values, so that backpropagation would work without additional changes.

- jacobian_tests_block_i – which sets the parameters of the i'th block 1st layer (in the additional two tests it is setting the 2nd and 3rd layer as well) with the given weights matrix and bias vector, and performs a forward pass from the i'th block. The function computes and returns the loss of this pass.

The derivatives verification tests are the same as in the previous section (standard NN).

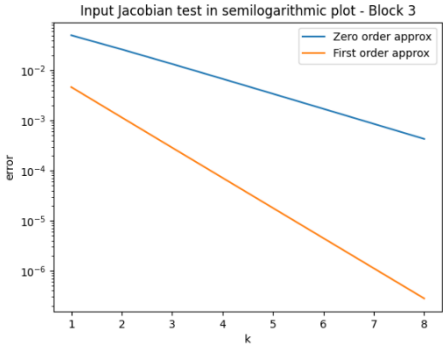
Note: I have not calculated the derivatives w.r.t the input inside the blocks, since this requires to alter the residual network enormously, to the point it barely represent the actual network.

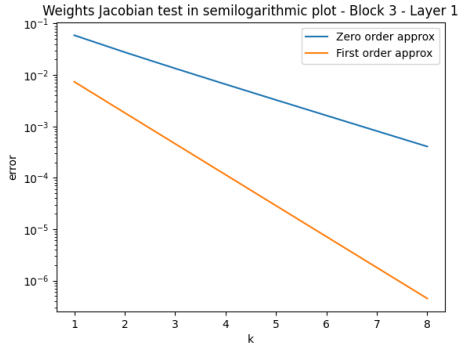
I'll denote the following elements in the network as follows:

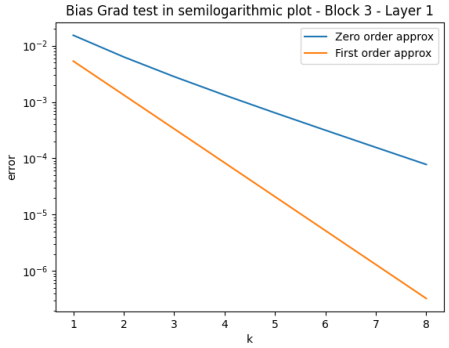
$$
\begin{aligned}
&Z_{1,1} = W_{1,1} * (X) \quad + b_{2,1}, && A_{1,1} = \tanh(Z_{1,1}) \\
&Z_{1,2} = W_{1,2} * (A_{1,1}) + b_{2,2}, && A_{1,2} = \tanh(Z_{1,2}) \\
&Z_{1,3} = W_{1,3} * (A_{1,2}) + b_{2,3}, && A_{1,3} = \tanh(Z_{1,3}), \qquad A_1 = A_{1,3} + X \\
&Z_{2,1} = W_{2,1} * (A_1) \quad + b_{2,1}, && A_{2,1} = \tanh(Z_{2,1}) \\
&Z_{2,2} = W_{2,2} * (A_{2,1}) + b_{2,2}, && A_{2,2} = \tanh(Z_{2,2}) \\
&Z_{2,3} = W_{2,3} * (A_{2,2}) + b_{2,3}, && A_{2,3} = \tanh(Z_{2,3}), \qquad A_2 = A_{2,3} + A_1 \\
&Z_{out} = W_{3,1} * (A_2) \quad + b_{3,1}, && \hat{Y} = \tanh(Z_{out}) \\
&(W_{out} = W_{3,1}, \qquad b_{out} = b_{3,1}, \qquad Z_{out} = Z_{3,1})
\end{aligned}
$$

Ben Gurion University of the Negev
Din Yair Sadot
302211917

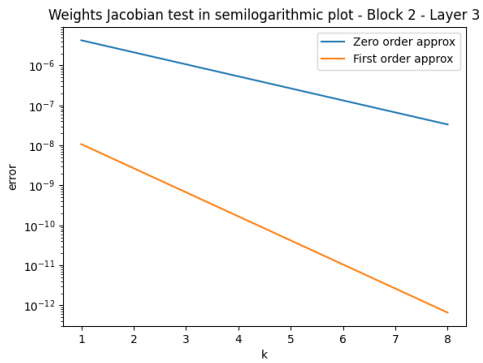
1. $\frac{\partial \mathcal{L}}{\partial Z_{out}} = \frac{1}{N}(\hat{Y} - Y)$
2. $\frac{\partial \mathcal{L}}{\partial W_{out}} = \frac{\partial \mathcal{L}}{\partial Z_{out}} \frac{\partial Z_{out}}{\partial W_{out}} = \frac{\partial \mathcal{L}}{\partial Z_{out}} * A_2^t$
3. $\frac{\partial \mathcal{L}}{\partial b_{out}} = \frac{\partial \mathcal{L}}{\partial Z_{out}} \frac{\partial Z_{out}}{\partial b_{out}} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_{out}}$ (average over columns)
4. $\frac{\partial \mathcal{L}}{\partial A_2} = \frac{\partial \mathcal{L}}{\partial Z_{out}} \frac{\partial Z_{out}}{\partial A_2} = \frac{\partial \mathcal{L}}{\partial Z_{out}} * W_{out} = W_{out}^t * \frac{\partial \mathcal{L}}{\partial Z_{out}}$ (This gradient is saved as residual)

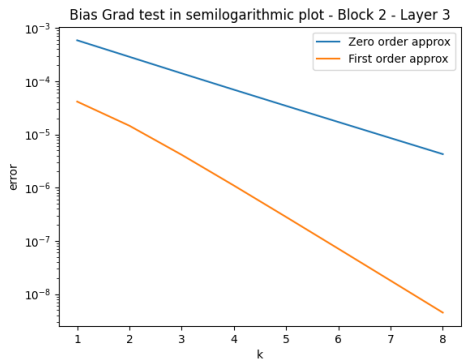






5. $\frac{\partial \mathcal{L}}{\partial Z_{2,3}} = \frac{\partial \mathcal{L}}{\partial A_2} \frac{\partial A_2}{\partial Z_{2,3}} = \frac{\partial \mathcal{L}}{\partial A_2} \odot (1 - tanh^2(Z_{2,3}))$
6. $\frac{\partial \mathcal{L}}{\partial W_{2,3}} = \frac{\partial \mathcal{L}}{\partial Z_{2,3}} \frac{\partial Z_{2,3}}{\partial W_{2,3}} = \frac{\partial \mathcal{L}}{\partial Z_{2,3}} * A_{2,2}^t$
7. $\frac{\partial \mathcal{L}}{\partial b_{2,3}} = \frac{\partial \mathcal{L}}{\partial Z_{2,3}} \frac{\partial Z_{2,3}}{\partial b_{2,3}} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_{2,3}}$

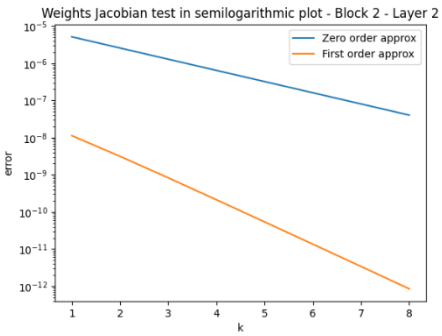


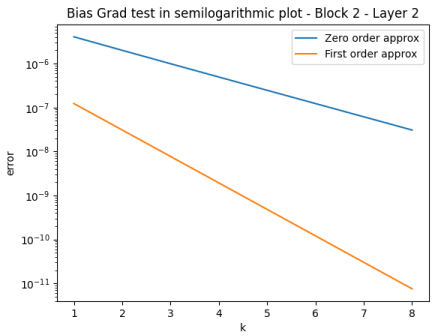


8. $\frac{\partial \mathcal{L}}{\partial A_{2,2}} = \frac{\partial \mathcal{L}}{\partial Z_{2,3}} \frac{\partial Z_{2,3}}{\partial A_{2,2}} = \frac{\partial \mathcal{L}}{\partial Z_{2,3}} * W_{2,3} = W_{2,3}^t * \frac{\partial \mathcal{L}}{\partial Z_{2,3}}$
9. $\frac{\partial \mathcal{L}}{\partial Z_{2,2}} = \frac{\partial \mathcal{L}}{\partial A_{2,2}} \frac{\partial A_{2,2}}{\partial Z_{2,2}} = \frac{\partial \mathcal{L}}{\partial A_{2,2}} \odot (1 - tanh^2(Z_{2,2}))$
10. $\frac{\partial \mathcal{L}}{\partial W_{2,2}} = \frac{\partial \mathcal{L}}{\partial Z_{2,2}} \frac{\partial Z_{2,2}}{\partial W_{2,2}} = \frac{\partial \mathcal{L}}{\partial Z_{2,2}} * A_{2,1}^t$

Ben Gurion University of the Negev
Din Yair Sadot
302211917

11. $\frac{\partial \mathcal{L}}{\partial b_{2,2}} = \frac{\partial \mathcal{L}}{\partial Z_{2,2}} \frac{\partial Z_{2,2}}{\partial b_{2,2}} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_{2,2}}$



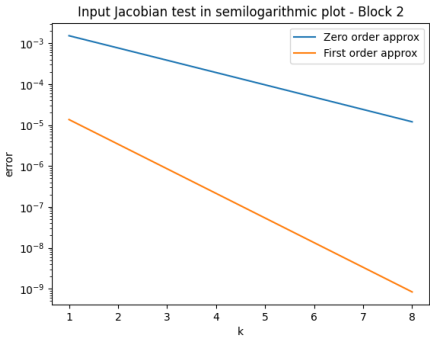


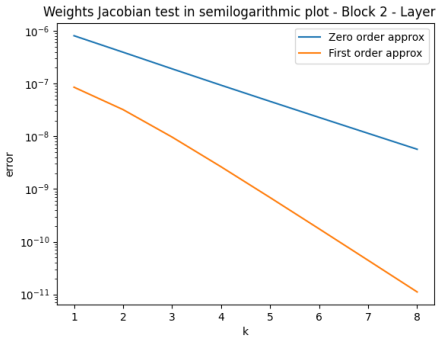
12. $\frac{\partial \mathcal{L}}{\partial A_{2,1}} = \frac{\partial \mathcal{L}}{\partial Z_{2,2}} \frac{\partial Z_{2,2}}{\partial A_{2,1}} = \frac{\partial \mathcal{L}}{\partial Z_{2,2}} * W_{2,2} = W_{2,2}^t * \frac{\partial \mathcal{L}}{\partial Z_{2,2}}$

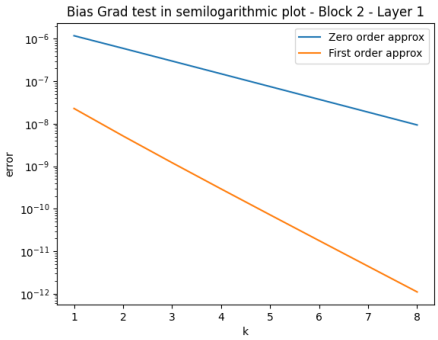
13. $\frac{\partial \mathcal{L}}{\partial Z_{2,1}} = \frac{\partial \mathcal{L}}{\partial A_{2,1}} \frac{\partial A_{2,1}}{\partial Z_{2,1}} = \frac{\partial \mathcal{L}}{\partial A_{2,1}} \odot (1 - tanh^2(Z_{2,1}))$

14. $\frac{\partial \mathcal{L}}{\partial W_{2,1}} = \frac{\partial \mathcal{L}}{\partial Z_{2,1}} \frac{\partial Z_{2,1}}{\partial W_{2,1}} = \frac{\partial \mathcal{L}}{\partial Z_{2,1}} * A_1^t$

15. $\frac{\partial \mathcal{L}}{\partial b_{2,1}} = \frac{\partial \mathcal{L}}{\partial Z_{2,1}} \frac{\partial Z_{2,1}}{\partial b_{2,1}} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_{2,1}}$ (Average on columns/samples)





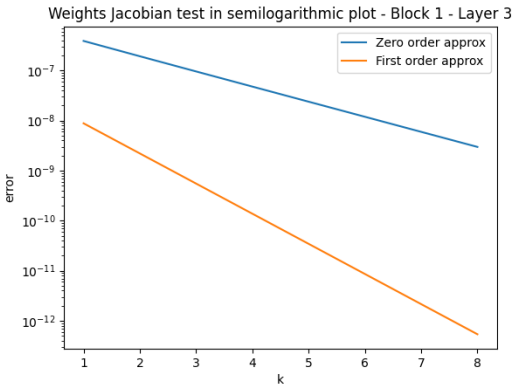


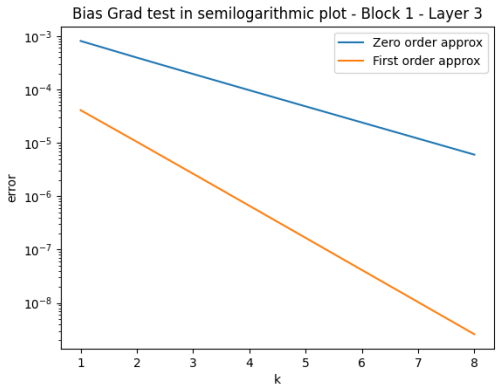
16. $\frac{\partial \mathcal{L}}{\partial A_1} = \frac{\partial \mathcal{L}}{\partial Z_{2,1}} \frac{\partial Z_{2,1}}{\partial A_1} = \frac{\partial \mathcal{L}}{\partial Z_{2,1}} * W_{2,1} + residual = W_{2,1}^T * \frac{\partial \mathcal{L}}{\partial Z_{out}} + \frac{\partial \mathcal{L}}{\partial A_2}$

17. $\frac{\partial \mathcal{L}}{\partial Z_{1,3}} = \frac{\partial \mathcal{L}}{\partial A_1} \frac{\partial A_1}{\partial Z_{1,3}} = \frac{\partial \mathcal{L}}{\partial A_1} \odot (1 - tanh^2(Z_{1,3}))$

18. $\frac{\partial \mathcal{L}}{\partial W_{1,3}} = \frac{\partial \mathcal{L}}{\partial Z_{1,3}} \frac{\partial Z_{1,3}}{\partial W_{1,3}} = \frac{\partial \mathcal{L}}{\partial Z_{1,3}} * A_{1,2}^t$

19. $\frac{\partial \mathcal{L}}{\partial b_{1,3}} = \frac{\partial \mathcal{L}}{\partial Z_{1,3}} \frac{\partial Z_{1,3}}{\partial b_{1,3}} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_{1,3}}$





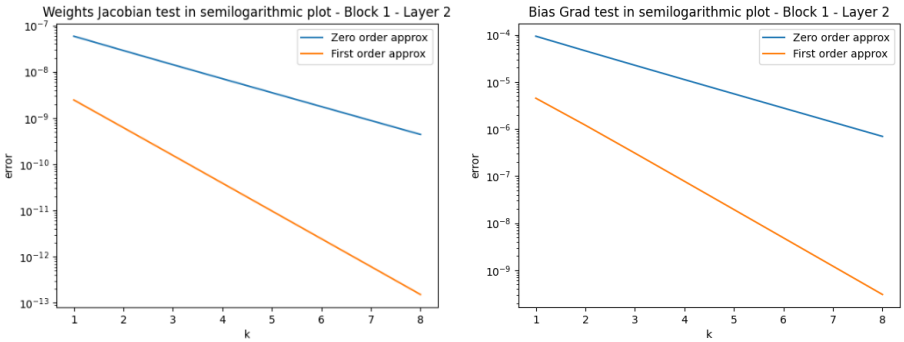
Ben Gurion University of the Negev
Din Yair Sadot
302211917

20. $\frac{\partial \mathcal{L}}{\partial A_{1,2}} = \frac{\partial \mathcal{L}}{\partial Z_{1,3}} \frac{\partial Z_{1,3}}{\partial A_{1,2}} = \frac{\partial \mathcal{L}}{\partial Z_{1,3}} * W_{1,3} = W_{1,3}^t * \frac{\partial \mathcal{L}}{\partial Z_{1,3}}$

21. $\frac{\partial \mathcal{L}}{\partial Z_{1,2}} = \frac{\partial \mathcal{L}}{\partial A_{1,2}} \frac{\partial A_{1,2}}{\partial Z_{1,2}} = \frac{\partial \mathcal{L}}{\partial A_{1,2}} \odot (1 - tanh^2(Z_{1,2}))$

22. $\frac{\partial \mathcal{L}}{\partial W_{1,2}} = \frac{\partial \mathcal{L}}{\partial Z_{1,2}} \frac{\partial Z_{1,2}}{\partial W_{1,2}} = \frac{\partial \mathcal{L}}{\partial Z_{1,2}} * A_{1,1}^t$

23. $\frac{\partial \mathcal{L}}{\partial b_{1,2}} = \frac{\partial \mathcal{L}}{\partial Z_{1,2}} \frac{\partial Z_{1,2}}{\partial b_{1,2}} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_{1,2}}$

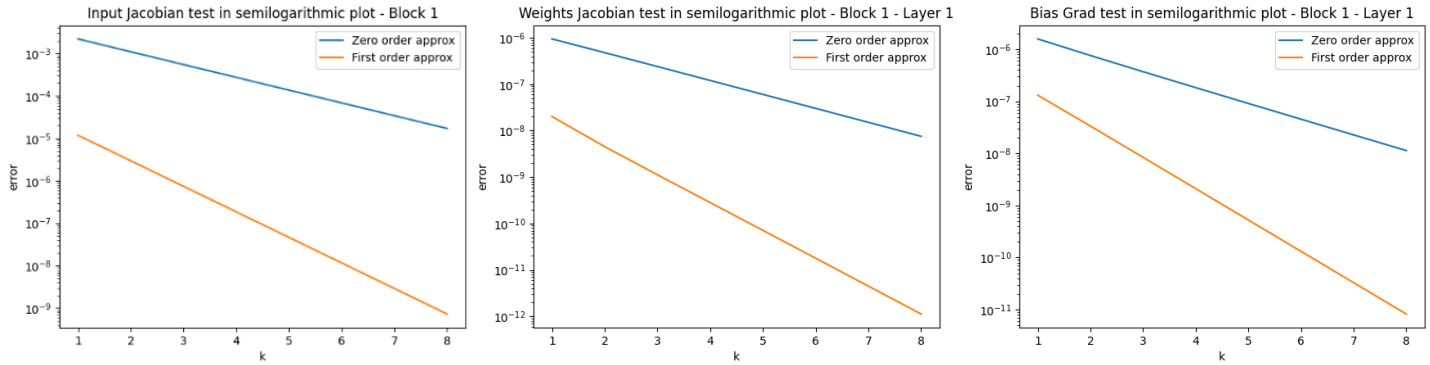


24. $\frac{\partial \mathcal{L}}{\partial A_{1,1}} = \frac{\partial \mathcal{L}}{\partial Z_{1,2}} \frac{\partial Z_{1,2}}{\partial A_{1,1}} = \frac{\partial \mathcal{L}}{\partial Z_{1,2}} * W_{1,2} = W_{1,2}^t * \frac{\partial \mathcal{L}}{\partial Z_{1,2}}$

25. $\frac{\partial \mathcal{L}}{\partial Z_{1,1}} = \frac{\partial \mathcal{L}}{\partial A_{1,1}} \frac{\partial A_{1,1}}{\partial Z_{1,1}} = \frac{\partial \mathcal{L}}{\partial A_{1,1}} \odot (1 - tanh^2(Z_{1,1}))$

26. $\frac{\partial \mathcal{L}}{\partial W_{1,1}} = \frac{\partial \mathcal{L}}{\partial Z_{1,1}} \frac{\partial Z_{1,1}}{\partial W_{1,1}} = \frac{\partial \mathcal{L}}{\partial Z_{1,1}} * X^t$

27. $\frac{\partial \mathcal{L}}{\partial b_{1,1}} = \frac{\partial \mathcal{L}}{\partial Z_{1,1}} \frac{\partial Z_{1,1}}{\partial b_{1,1}} = \Sigma \frac{\partial \mathcal{L}}{\partial Z_{1,1}}$ (Average on columns/samples)



Ben Gurion University of the Negev
Din Yair Sadot
302211917

Whole Network Gradient Test

In the previous sections of part II, I have tested each layer of my neural network and have verified that each layer's derivatives with respect to the inputs, weights and biases are correct. In this section I have performed a gradient verification for the parameters of the whole network in a single test. For convenience I have defined all the layers of the network for this test as the same size, meaning that each layer has equal size (N) of neurons.

Note: In this section I have changed the backpropagation process of the standard neural network into a loop, instead of manually computing each layer's derivatives as in section 2.1. This is because this section requires that the network consists of L layers, which means that the number of layers is unknown during implementation. I have tested the altered implementation with the tests I have implemented in section 2.1 and got the same results, which means it is correctly implemented.

To perform this test, I have implemented 2 auxiliary functions:

- initialize_by_vector (vector, N) – this function gets a parameter vector, which contains weights and biases for the whole network, flattened and ordered sequentially: $(W_1, b_1, W_2, \dots, W_{L-1}, b_{L-1})$. The functions also take parameter N, which represents the number of neurons in any layer of this network. The function slices up the vector accordingly and initializes weights matrices and bias vectors for the whole network.

- flatten_gradients (grads) – this function takes the list of tuples of weights and biases which is the return value from the backpropagation. The function flattens the whole list into one 1-D vector and returns it.

- whole_network_test (X, Y, vec, N) – this function initializes a network by the vector vec, performs a forward pass and computes the loss. The function returns the loss and the activations and pre-activations returned from forward pass, for the usage of backpropagation.

- whole_network_gradient (X, Y, vec, N) – this function performs backpropagation and returns a flattened version of the results.

I have implemented corresponding function in the ResNet, which suits the structure of blocks.

The actual gradient test is implemented in a file called "whole_network_test.py" and follows the same conventions of the gradient tests that were performed before in this project.

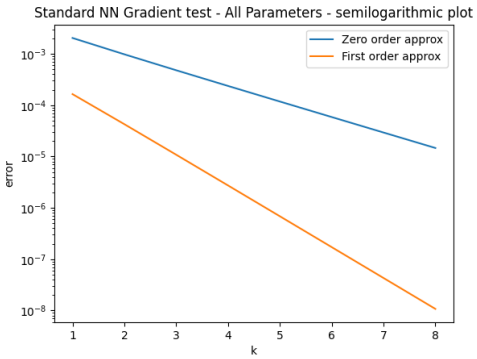
Ben Gurion University of the Negev
Din Yair Sadot
302211917

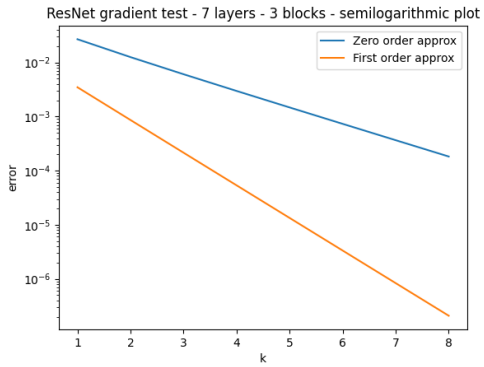
In the graphs below, you can see that the error orders obey the Taylor's expansions orders (which were mentioned before in this project), and that verifies that the derivatives of the whole network are correct:

- $F(vec)$ = whole_network_test(X, Y, vec, N)
  g_F = whole_network_gradient

- Zero order approx:
  $F(vec + \varepsilon d) - F(vec)$

- First order approx:
  $F(vec + \varepsilon d) - F(vec) - g_F * \varepsilon d$





Ben Gurion University of the Negev
Din Yair Sadot
302211917

Training the neural networks

After verifying that both my standard and my residual neural networks are built correctly, and all the derivatives are correct, I have begun to train the networks on the Peaks data set, which I have used earlier in section 2.1.3 when the network was consisted of only the output layer.

At first, the results were disappointing. I have trained both networks and got poor accuracy rates (about 25%), even when I tried a few different hyper parameters' values. The loss value was almost completely unchanged between epochs.

This issue was eventually solved by 3 important steps:

- I have changed the number of neurons in the layers significantly – at first, I have tried networks of between 4-8 neurons per layer. The network began to produce better results when I changed the number of neurons per layer to 32, 64 and 128 (at descending order, where the higher number of neurons are at the beginning of the network).

- I have changed the activation function from tanh to ReLU, which showed much better results at converging and decreasing the loss.

- I have changed the initialization of the weights – from regular normal distribution random values, to He (Kaiming) initialization which corresponds to the following formula: $W \sim N(0, \frac{2}{n_{input}})$.
  This step has tremendously changed the training results and improved the accuracy to about 90%.

After the network has shown that it can converge, I tried a few networks' lengths. I have seen that short networks did not reach a certain accuracy no matter what the other parameters were. In addition, too long networks did not improve the accuracy and made the training much longer. I have got the best results for network of 4 hidden layers of 16-64 neurons each (in decreasing order from the input layer to the output layer).

I have used a constant learning rate and seen that too large learning rate makes the improvement very unstable, but too small learning rate makes the training process very long and that requires more epochs. I got the best results with a learning rate of 0.001.

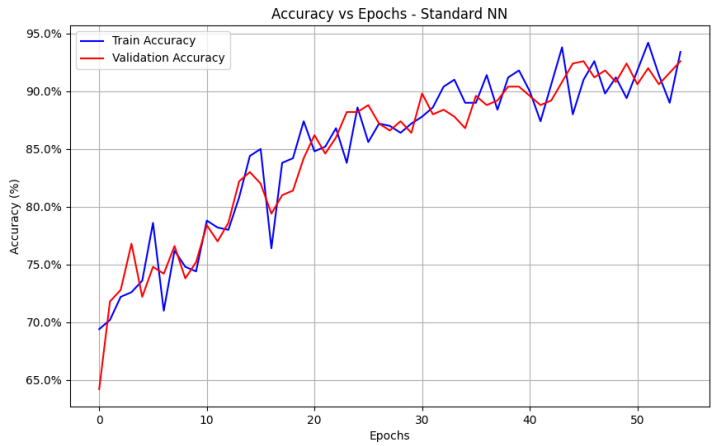
The size of the mini batch has also affected the maximum accuracy rate that the network can achieve. A larger size of batch has converged quickly, and the training was faster, but the accuracy was slightly lower than with smaller batches. I got the best results with mini batches of **16** samples.

Finally, I have run the training for large numbers of epochs and picked the number of epochs which have managed to get a good, stabilized accuracy, and which have stopped improving upon more epochs. I have gotten the best results at around 55 epochs.

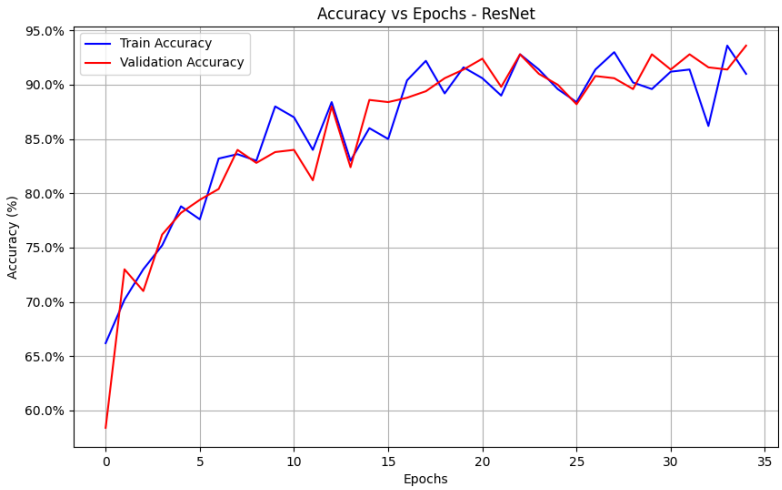
With the settings I have described here I have reached an average of 92.5% accuracy.

Ben Gurion University of the Negev
Din Yair Sadot
302211917

The following graph describes the accuracy in terms of the number of epochs, with the hyper parameters values I have described above, for the "**Peaks**" dataset:

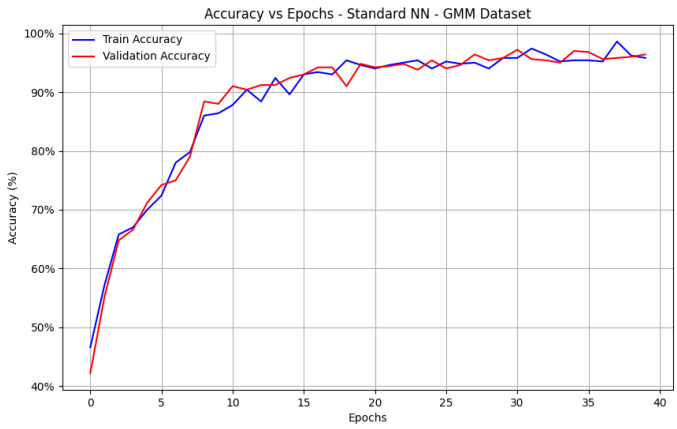


With the ResNet I have got very similar results, with 3 blocks of 3 layers which consists of [64, 32, 32, 5] neurons, with a smaller learning rate (0.0004), and with less epochs (35):

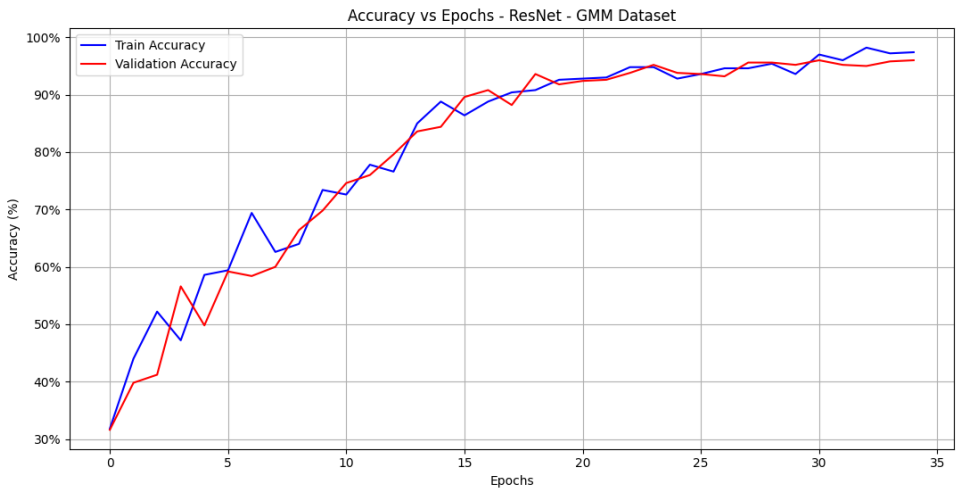


Ben Gurion University of the Negev
Din Yair Sadot
302211917

For the "**GMM**" dataset I have changed the parameters just a little bit. I have used 5 hidden layers of [64, 32, 16, 8 ,8] number of neurons, learning rate of 0.005 and 40 epochs, and mini batches of 8 samples for best performance in the standard neural network. With these settings I have reached an average accuracy of 95%. The following graph plots the accuracy in terms of the number of epochs for the "**GMM**" dataset:



In the ResNet I set 3 hidden layers' blocks of [32, 16, 8] neurons, learning rate of 0.00035, 35 epochs and mini batches of 16 samples, and got an accuracy of 96%. The results are presented in the following graph:



Ben Gurion University of the Negev
Din Yair Sadot
302211917

Reducing the cost of training

Since the training process is expensive in calculations, I tried to get the maximum accuracy while using less than $100 * C$ trainable parameters. Since I am training the Peaks data set, which has 5 classes, I could use as much as 500 parameters. The parameters include the weights and the biases.

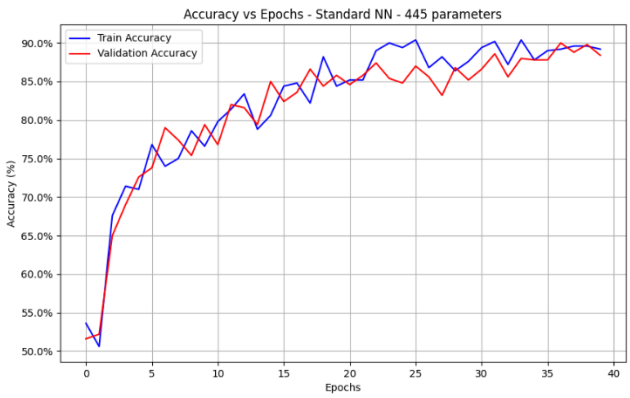
The parameter count is as follows: for each layer we compute the size of the weights matrix by multiplying the size of the previous layer with the size of the output layer, and add the size of the bias vector, or in general: output size * (input size + 1).

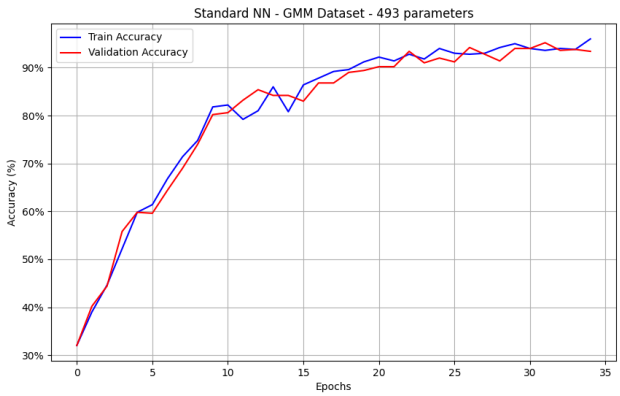
I found that a longer network works better than a wider network. A wide layer requires a very large weight matrix, which is why I used only 8-16 neurons per layer (in descending order from the input layer). I found that using powers of 2 is still a better strategy than using other numbers, even if using other numbers results in more parameters (closer to the 500 limit).

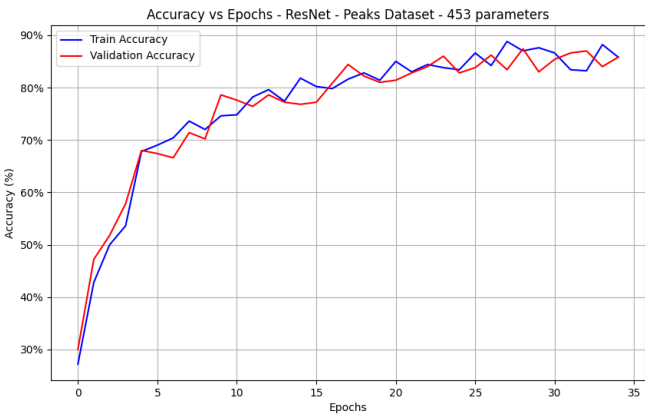
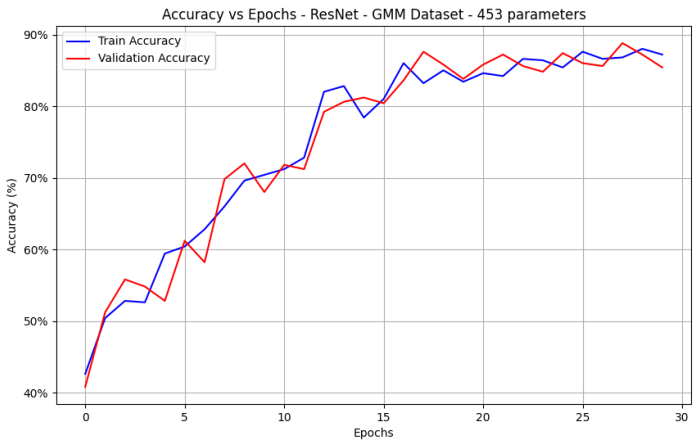
So eventually I used a network of 5 hidden layers, ordered as such: [16, 8, 8, 8, 8]. The output layer always consists of 5 neurons since the dataset is classified into 5 classes.

Also, I noticed that since the layers are narrower, it works better with smaller mini batches, so I reduced the size of the batch to 8 at a time. Also, I reduced the learning rate a bit – to 0.006. I reduced the number of epochs to 40 since I noticed the accuracy does not get better upon more iterations. I have used these settings for both datasets and got good performance.

The results were around 90% of accuracy for the "Peaks" dataset, and 93% for the "GMM" dataset, which is pretty close to my best results overall.
The results are plotted in the following graph:





Ben Gurion University of the Negev
Din Yair Sadot
302211917

In the ResNet I have found that 2 hidden blocks of 8 neurons for each layer (which equals 453 parameters) makes the best performance overall. I have set the number of epochs to 30, the batch size to 8, and the learning rate to 0.0004 and got to an average accuracy of 88% on the GMM dataset. On the Peaks dataset I have changed the number of epochs to 35 and got to an average accuracy of 86%. The results are presented in the following graphs: